

RNN

RNN has trouble with Vanishing / exploding gradients during back propagation

→ hard to calculate deep layer

* Exploding Gradient - use Gradient clipping.

* Vanishing Gradients → Alternative architecture.

① LSTM - Long Short-Term Memory. (LSTM, 1997)

② Gated Recurrent Unit (GRU, 2014)

Basic Idea: Need some Memory.

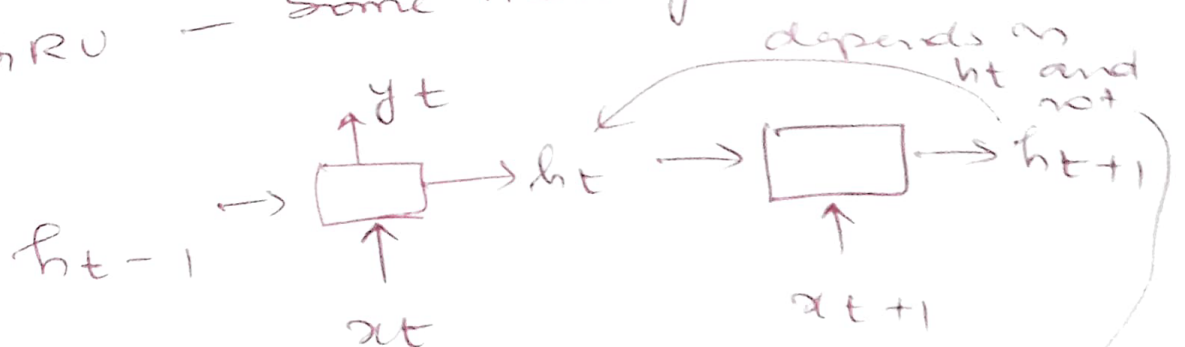
* Short term Memory →

"Remembering the articles what you have seen some times earlier"

A dog eats well but some times he vomited.

LSTM - separate cell for memory.

GRU - Some memory like operation.

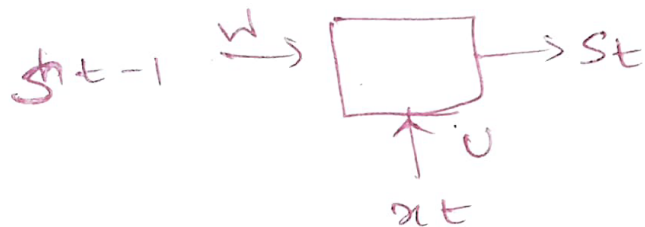


h_t stores some part of h_{t-1} to do with h_{t+1}

explicitly in h_{t-1}

LSTM and GRU stores some part of previous computation and output.

Simplified GRU



$$s_t = \phi(\underbrace{w s_{t-1} + w x_t}_{z_g}) \quad [\text{o/p of RNN}]$$

↓

$$g = \phi(z_g) \quad z_g = w s_{t-1} + w x_t$$

(or) Interpolation

GRU = linear combination of ~~RNN~~ RNN and older computation

$$s_t = (1 - \lambda) g + \lambda h_{t-1}$$

$$\lambda \in [0, 1]$$

when $\lambda = 1$ ~~st~~ $s_t = s_{t-1}$ [pure and simple memory]

$\lambda = 0$ $s_t = g \rightarrow$ RNN output.

Simplified GRU.

$$s_t = (1 - f) \odot g + f \odot h_{t-1}$$

↓
forget gate

$$h_t = (1-f) \odot g + f \odot h_{t-1}$$

↑
vector.

$$f \in [0, 1]$$

what value of 'f' we choose?

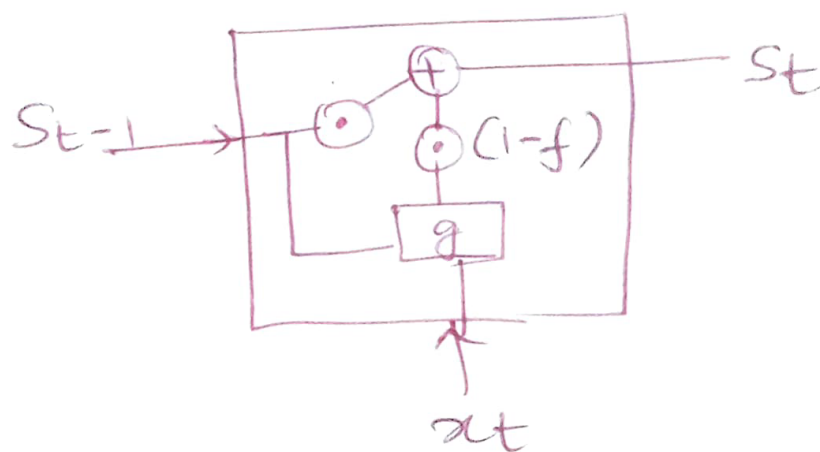
↳ Parameter to be trained.

$$f = \sigma(W_f s_{t-1} + u_f x_t)$$

$$s_t = (1-f) \odot g + f \odot h_{t-1}$$

$$g = \phi(W_g s_{t-1} + u_g x_t) = \phi(z_g)$$

$$f = \sigma(W_f s_{t-1} + u_f x_t) = \sigma(z_f)$$



(update)
remember
gate

Full Gated Recurrent Unit

Works
much better
for deeper
networks.

$$s_t = (1-f) \odot g' + f \odot h_{t-1}$$

$$g' = \phi(z_g); \quad z_g = W_g (x \odot s_{t-1}) + u_g x_t$$

$$f = \sigma(z_f); \quad z_f = (W_f s_{t-1} + u_f x_t)$$

$$r = \sigma(z_r); \quad z_r = (W_r h_{t-1} + u_r x_t)$$